Лабораторная работа №3

Объектно-ориентированное программирование в Python

1. Цель работы

Научиться создавать классы и объекты, использовать наследование, инкапсуляцию, композицию и полиморфизм.

2. Постановка индивидуального задания

Вариант 8. Ресторан. Создать базовый класс MenuItem и производные классы Dish и Drink. Для блюда хранить калорийность, для напитка — объём. Реализовать расчёт итоговой стоимости с учётом особенностей типа товара и класс Order.

3. Краткие теоретические сведения

Класс описывает общие свойства и поведение объектов. Наследование позволяет специализировать базовый класс. Абстрактный метод final_price() задаёт общий интерфейс, а переопределение в Dish и Drink демонстрирует полиморфизм. Order использует композицию и хранит позиции заказа.

4. Исходный код программы

Файл main.py

```
from abc import ABC, abstractmethod

class MenuItem(ABC):
    def __init__(self, name, price):
        if not name.strip() or price < 0:
            raise ValueError("Название не должно быть пустым, цена — неотрицательной.")
        self.name = name
        self.price = float(price)

    @abstractmethod
    def final_price(self, quantity=1):
        pass

    def describe(self):
```

```
        return f"{self.name}: {self.price:.2f} руб."

class Dish(MenuItem):
    def __init__(self, name, price, calories):
        super().__init__(name, price)
        if calories < 0: raise ValueError("Калорийность не может быть отрицательной.")
        self.calories = int(calories)

    def final_price(self, quantity=1):
        return self.price * quantity

    def describe(self):
        return f"Блюдо: {self.name}, {self.calories} ккал, {self.price:.2f} руб."

class Drink(MenuItem):
    def __init__(self, name, price, volume_ml):
        super().__init__(name, price)
        if volume_ml <= 0: raise ValueError("Объём должен быть больше нуля.")
        self.volume_ml = int(volume_ml)

    def final_price(self, quantity=1):
        # Для напитка большого объёма действует скидка 5% при заказе от 3 шт.
        total = self.price * quantity
        return total * 0.95 if self.volume_ml >= 500 and quantity >= 3 else total

    def describe(self):
        return f"Напиток: {self.name}, {self.volume_ml} мл, {self.price:.2f} руб."

class Order:
    def __init__(self, number, customer):
        self.number = number
        self.customer = customer
        self.items = []

    def add_item(self, item, quantity=1):
        if quantity <= 0: raise ValueError("Количество должно быть больше нуля.")
        self.items.append((item, quantity))

    def total(self):
        return sum(item.final_price(quantity) for item, quantity in self.items)

    def show(self):
        print(f"Заказ №{self.number}, клиент: {self.customer}")
        for item, quantity in self.items:
            print(f"  {item.describe()} x {quantity}")
        print(f"Итого: {self.total():.2f} руб.")

def main():
    soup = Dish("Суп овощной", 240, 180)
    pasta = Dish("Паста", 390, 520)
    water = Drink("Вода", 90, 500)
    order = Order(1, "Куршаков Е.И.")
    order.add_item(soup, 1); order.add_item(pasta, 1); order.add_item(water, 3)
    print("=== Меню ===")
    for item in (soup, pasta, water): print(item.describe())
    print("\n=== Заказ ===")
    order.show()
```

```
if __name__ == "__main__": main()
```

5. Объяснение работы программы

MenuItem проверяет название и цену. Dish добавляет калорийность и рассчитывает цену как стоимость за единицу, Drink хранит объём и предоставляет скидку для заказа нескольких больших напитков. Order хранит пары товар–количество и вычисляет общую сумму.

6. Примеры работы программы в виде снимков экрана

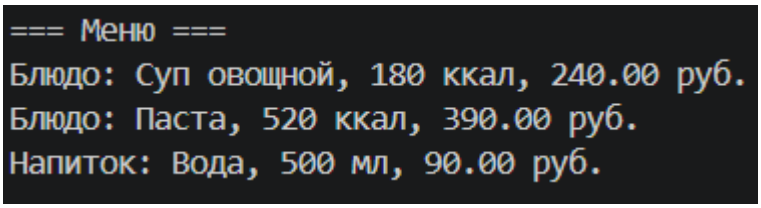


Рисунок 3.1 – Отображение блюд и напитков с их характеристиками.

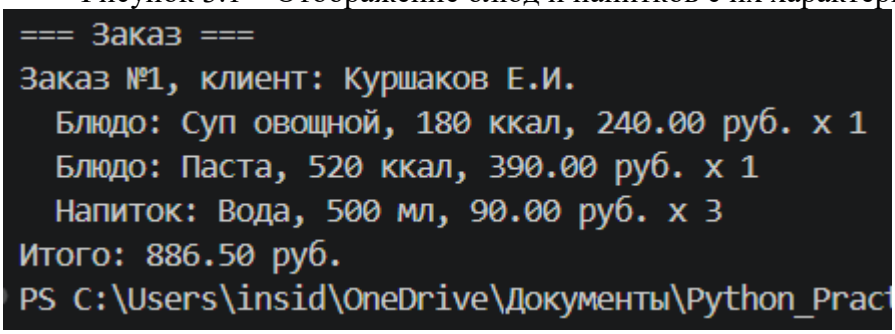


Рисунок 3.2 – Формирование заказа с несколькими позициями.

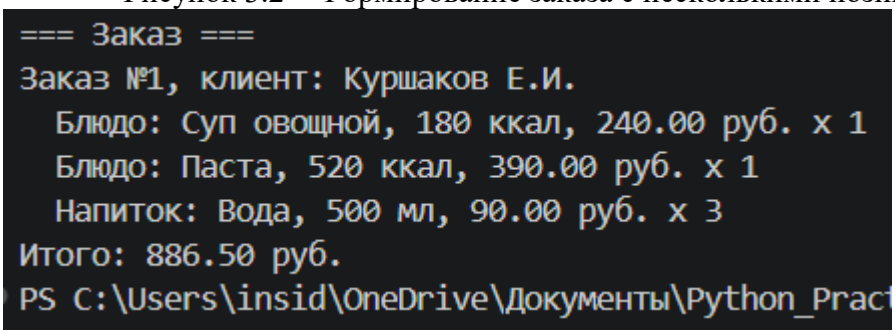


Рисунок 3.3 – Расчёт итоговой стоимости с учётом особенностей напитков.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. MenuItem проверяет название и цену. Dish добавляет калорийность и рассчитывает цену как стоимость за единицу, Drink хранит объём и предоставляет скидку для заказа нескольких больших напитков. Order хранит пары товар–количество и вычисляет общую сумму. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич